

A.D. 1308 unipg
DIPARTIMENTO
DI INGEGNERIA

Tesina Finale di

Data Security and Blockchain

Corso di Laurea in Ingegneria Informatica ed Elettronica – A.A. 2023-2024

Dipartimento di Ingegneria

docente

Prof. Luca Grilli

Bakery Shop NFT

frontend-only Web DApp running on Sepolia Ethereum Testnet
technology: HTML/CSS/JavaScript/Web3.js



studente

361600 **Aurora** **Traversini** aurora.traversini@studenti.unipg.it

Data ultimo aggiornamento: 21 ottobre 2026

0. Indice

1 Descrizione del Problema **2**
1.1 Lo standard ERC-721 2
1.1.1 ERC721URIStorage 4
1.2 Tecnologie utilizzate 4

2 Specifica dei Requisiti **6**

3 Bakery Shop NFT **9**
3.1 Architettura del Sistema Software 9
3.1.1 Componenti dell'Architettura 10
3.2 Smart contract 10
3.3 Integrare lo Smart contract con il Front End 12

4 Esempio d'uso **15**
4.1 Sezione Home: Totale dei prodotti e Restock 17
4.2 Sezione Products 20
4.3 Sezione About 25
4.4 Sezione Latest Clients 26
4.5 Sezione Contact 27

5 Problemi Riscontrati **28**

6 Sviluppi futuri e Conclusioni **30**

7 Bibliografia **32**

8 Appendice **33**
8.1 Scelta dei metadati per rendere unici i token NFT 33

1. Descrizione del Problema

Tale progetto ha lo scopo di sviluppare una Web DApp, denominata *Bakery Shop NFT*, finalizzata alla creazione e all'acquisto di prodotti unici di una panetteria, rappresentati come *Non Fungible Token* (*NFT*) conformi allo standard ERC-721 [9] della blockchain Ethereum.

L'obiettivo primario è rendere questa esperienza accessibile anche a utenti non esperti, consentendo loro di interagire con la blockchain e creare i propri NFT senza la necessità di utilizzare ambienti di sviluppo complessi. Infatti l'interfaccia semplice e intuitiva dell'applicazione permette la creazione e l'acquisto di NFT, consentendo agli utenti di partecipare al mercato dei token unici senza richiedere conoscenze approfondite della tecnologia blockchain, ma utilizzando semplici moduli di input HTML, direttamente dal browser web.

La Web DApp Bakery Shop NFT sarà eseguita interamente nel browser utilizzando tecnologie standard del web come HTML, CSS e JavaScript, mentre l'accesso attraverso il browser è consentito dall'utilizzo di un server web locale, mediante l'estensione *Live Server* di Visual Studio Code.

Per quanto concerne l'interazione con la blockchain Ethereum, la DApp farà utilizzo della libreria *Web3.js* [6]. Gli smart contract della DApp saranno eseguiti nella testnet pubblica Sepolia Ethereum [7].

Di seguito sarà data una breve descrizione dello standard ERC-721, dopodiché si fornirà una descrizione generale della Web DApp Bakery Shop che si intende realizzare.

1.1 Lo standard ERC-721

Gli NFT, o token non fungibili, sono un tipo di token crittografico che rappresenta qualcosa di unico e indivisibile, garantendo la proprietà digitale o fisica di beni unici e non riproducibili all'interno di un ecosistema decentralizzato.



Figura 1.1: NFT, Non Fungible Token

Lo standard ERC-721 è stato introdotto per definire uno standard comune per la creazione e la gestione di NFT sulla blockchain Ethereum. Esso stabilisce le specifiche tecniche e i requisiti per i contratti intelligenti che gestiscono NFT, garantendo l'interoperabilità e l'intercambiabilità tra diverse piattaforme e applicazioni.

Di seguito sono riportate alcune delle caratteristiche principali di ERC-721:

- **Non Fungibilità**: Gli NFT conformi allo standard ERC-721 sono non fungibili, il che significa che ogni token è unico e indivisibile.
- **Unicità e indivisibilità**: Ogni token ERC-721 ha un identificatore univoco, chiamato token ID, che lo distingue dagli altri token nella stessa collezione.
- **Trasferimento di proprietà**: Il trasferimento di un NFT da un indirizzo Ethereum a un altro è supportato dallo standard ERC-721.
- **Eventi**: Gli smart contract conformi allo standard ERC-721 possono emettere eventi per notificare le parti interessate riguardo alle transazioni e ai cambiamenti di stato del NFT.

I token creati secondo lo standard ERC-721 hanno molteplici utilizzi, tra cui i più comuni:

1. **Giochi e collezionabili digitali**: Molte piattaforme di gioco utilizzano NFT conformi a ERC-721 per rappresentare oggetti in-game, personaggi e carte collezionabili.
2. **Arte digitale e opere d'arte**: Gli artisti digitali possono creare e vendere opere d'arte digitali come NFT, garantendo la proprietà e l'autenticità delle loro opere sulla blockchain.

3. **Immobiliare virtuale**: Le piattaforme di realtà virtuale e di mondo virtuale utilizzano NFT per rappresentare terreni, edifici e proprietà virtuali, consentendo agli utenti di possedere e scambiare proprietà digitali.

La conformità allo standard ERC-721 assicura che gli NFT creati siano compatibili con una vasta gamma di applicazioni e piattaforme, consentendo agli utenti di creare, trasferire e visualizzare i loro beni digitali unici in modo sicuro e affidabile.

In questo progetto, lo standard ERC-721 viene utilizzato per creare e gestire dei token non fungibili che rappresentano prodotti unici della panetteria. Ogni prodotto della panetteria viene rappresentato da un NFT unico, che include metadati come il nome del prodotto, la descrizione, l'immagine e altre informazioni pertinenti. Gli utenti possono creare, acquistare e scambiare questi NFT sulla piattaforma Bakery Shop, partecipando così al mercato dei token unici sulla blockchain Ethereum.

1.1.1 ERC721URIStorage

Il contratto *BakeryShop.sol* implementa in realtà *ERC721URIStorage*, un'estensione del contratto principale *ERC721.sol*, che include le estensioni dello standard dei metadati (IERC721Metadata) e un meccanismo per i metadati per-token. È da qui che proviene il metodo *_setTokenURI* che sarà in seguito utilizzato per memorizzare i metadati di un token [1].

1.2 Tecnologie utilizzate

La Web DApp è costruita utilizzando le seguenti tecnologie:

- **HTML, CSS e JavaScript**: Per creare l'interfaccia utente e gestire l'interazione con la blockchain.
- **Web3.js**: Per l'interazione con la blockchain Ethereum e gli smart contract [6].
- **MetaMask**: un portafoglio elettronico locale multivaluta disponibile come estensione del browser Web. È uno strumento fondamentale per la sicurezza, poiché memorizza una passphrase segreta e le chiavi private per firmare le transazioni per spendere o trasferire criptovalute o token crittografici. E' stato utilizzato per gestire gli account Ethereum e autorizzare le transazioni.
- **RemixIDE**: un IDE online per lo sviluppo di contratti intelligenti su Ethereum o su blockchain compatibili con EVM nel linguaggio Solidity. È open-source e la sua crescente popolarità è legata al non richiedere alcuna installazione e configurazione per l'utilizzo. Inoltre, Remix IDE supporta l'intero

ciclo di sviluppo del software: modifica del codice sorgente, compilazione, debug, distribuzione e test [5].

- **Sepolia Testnet**: è una stretta emulazione della Mainnet di Ethereum. Per questo, è la testnet predefinita raccomandata per lo sviluppo di applicazioni. Esattamente come la Mainnet, la Testnet Sepolia ha la sua criptovaluta nativa, ovvero il Sepolia ETH, che è la controparte dell'ETH originale. Analogamente all'ETH, il Sepolia ETH viene utilizzato per pagare le commissioni di transazione sulla Sepolia Testnet [7].

- **Pinata**: un servizio di hosting che utilizza IPFS (InterPlanetary File System) per garantire la persistenza e l'accessibilità dei dati [4]. E' stato utilizzato per il caricamento e l'hosting dei metadati di ciascun singolo prodotto.

- **OpenSea Testnet**: è una versione di test della popolare piattaforma OpenSea, utilizzata per il trading di token NFT (Non-Fungible Tokens) su blockchain [8]. Questa piattaforma consente agli sviluppatori e agli utenti di sperimentare e verificare le funzionalità dei propri contratti smart e dei relativi NFT senza dover utilizzare la rete principale. Per visualizzare i token NFT mintati da un determinato contratto sulla Testnet, è sufficiente cercare l'indirizzo del contratto su OpenSea Testnet. Una volta trovato il contratto, è possibile vedere tutti i token NFT ad esso associati, insieme ai relativi metadati, che includono informazioni dettagliate come il nome del token, la descrizione, l'immagine e altre proprietà specifiche.

2. Specifica dei Requisiti

Di seguito verranno illustrati i principali requisiti previsti per il progetto *Bakery-Shop*.

1. Progettazione di uno smart contract conforme allo standard ERC721 implementato in Solidity.
2. Realizzazione di un interfaccia utente intuitiva e chiara, che non induca l'utilizzatore allerrore; tale interfaccia deve consentire di effettuare le operazioni implementate.
3. Linterfaccia deve essere quanto più fluida possibile.
4. Le notifiche e le informazioni devono essere presentate all'utente in modo chiaro e comprensibile, tramite apposite finestre di dialogo.
5. L'applicazione dovrà prevedere una finestra di dialogo durante l'attesa che una transazione venga completata.
6. Lapplicazione dovrà essere eseguibile in un qualsiasi browser recente, in una sola scheda.
7. L'applicazione dovrà consentire al proprietario di creare nuovi token ERC-721 rappresentativi dei prodotti della panetteria.
8. Ogni token creato dovrà essere unico e associato a un determinato prodotto, e avrà un ID univoco che lo identifichi.
9. Ciascun token sarà anche correlato ad un URI che indica lindirizzo di un file JSON contenente i metadati associati al token.
10. Ogni prodotto ha un nome, un prezzo, una quantità disponibile e una quantità massima (dato che i prodotti saranno presenti in qualità limitata).

11. Per garantire la sicurezza e la tracciabilità delle transazioni, l'applicazione dovrà prevedere un sistema di autenticazione degli utenti tramite lestensione browser MetaMask o inserendo manualmente la propria chiave privata. Questo permetterà di identificare chi sta effettuando determinate operazioni sulla piattaforma.

12. Alcune funzionalità saranno riservate unicamente al proprietario, *owner*), della panetteria.

13. Lutente deve convalidare ciascuna transazione destinata a modificare lo stato blockchain (come l'acquisto o il restock) manualmente tramite Metamask.

14. L'applicazione deve permettere agli utenti di acquistare prodotti dalla panetteria virtuale tramite l'interfaccia web, utilizzando Ether come metodo di pagamento.

15. Gli utenti devono avere una quantità di Ether sufficiente per acquistare i prodotti desiderati.

16. Dopo un acquisto riuscito, il sistema deve trasferire il prodotto all'utente e aggiornare dinamicamente la quantità disponibile.

17. Solo al proprietario è consentito di rifornire i prodotti esistenti nella panetteria, tramite un opportuno bottone.

18. Il proprietario deve specificare il tipo di prodotto e la quantità da rifornire.

19. Dopo il rifornimento, il sistema deve aggiornare la quantità disponibile dei prodotti.

20. A seguito di un rifornimento, il sistema deve consentire al proprietario di impostare l'URI del token NFT per ogni prodotto.

21. Gli utenti possono visualizzare le informazioni sui prodotti disponibili (inclusi nome, prezzo e quantità) tramite l'interfaccia web.

22. Gli utenti possono visualizzare il totale dei prodotti disponibili nel negozio in qualunque momento, tramite un opportuno bottone.

23. Il sistema deve condurre una lotteria automatica implementata nel contratto dopo ogni acquisto ben riuscito.

24. In caso di vincita nella lotteria, l'utente deve ricevere una colazione gratuita sotto forma di token NFT.

25. Il sistema deve notificare gli utenti sul risultato della lotteria tramite un opportuno dialog.

26. Il sistema deve essere affidabile nell'esecuzione delle transazioni e nella gestione dei rifornimenti, attraverso opportuni controlli.

3. Bakery Shop NFT

3.1 Architettura del Sistema Software

Viene ora illustrata la struttura del sistema software, in particolare la sua architettura.

Il progetto utilizza un'architettura client-side basata su JavaScript, Html e CSS. Il codice JavaScript interagisce con il contratto Ethereum *BakeryShop.sol* utilizzando la libreria *Web3.js*, che fornisce un insieme di funzionalità per comunicare con la blockchain Ethereum direttamente dal browser.

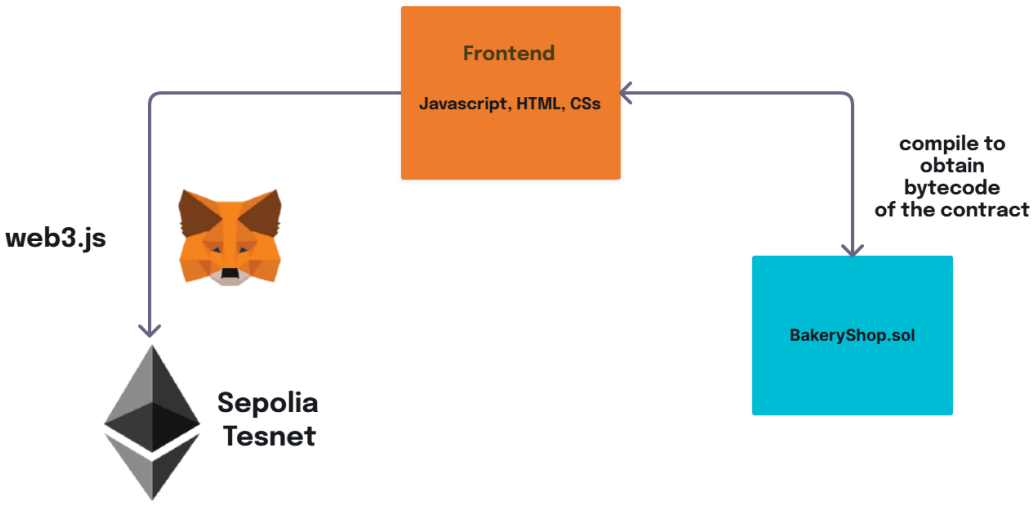


Figura 3.1: Architettura del sitema sviluppato.

Il contratto Ethereum, identificato dall'indirizzo specificato nella variabile *con-*

tractAddress, rappresenta il back-end dell'applicazione. Il codice JavaScript invia transazioni e legge i dati dal contratto utilizzando le funzioni fornite da Web3.js.

Il progetto integra anche il wallet *Metamask* per gestire le transazioni sulla blockchain Ethereum. Metamask è un'estensione del browser che permette agli utenti di eseguire transazioni Ethereum direttamente nel browser senza dover eseguire un nodo Ethereum completo [3]. Il progetto utilizza Metamask per autenticare gli utenti, per firmare e inviare transazioni alla blockchain Ethereum.

Per quanto riguarda l'ambiente di sviluppo, il progetto utilizza "*Live Server*", un'estensione di Visual Studio Code che lancia un server di sviluppo locale con live reload. Questo permette di visualizzare immediatamente le modifiche apportate al codice senza dover ricaricare manualmente la pagina.

Infine, il progetto utilizza il *Document Object Model (DOM)* per interagire con l'interfaccia utente dell'applicazione, leggendo i dati dagli input dell'utente e aggiornando l'interfaccia in risposta agli eventi.

3.1.1 Componenti dell'Architettura

Client-Side (Frontend):

- **Interfaccia Utente (UI)** = La parte visiva dell'applicazione accessibile tramite il browser Web (in questo caso si è utilizzato Microsoft Edge), dove gli utenti possono navigare tra i prodotti disponibili, effettuare acquisti e creare nuovi prodotti NFT.
- **Script JavaScript:** = Gestisce gli eventi dell'interfaccia utente e ne aggiorna dinamicamente il contenuto. Utilizza la libreria Web3.js per interagire con il contratto intelligente Bakery Shop sulla blockchain Ethereum.
- **Web3.js**: Consente al frontend di comunicare con la blockchain Ethereum. Gestisce la creazione, l'acquisto e la trasferimento di prodotti NFT utilizzando le funzioni del contratto intelligente *BakeryShop.sol*.

Backend

- **BakeryShop.sol**: Lo smart contract sviluppato in Remix IDE.

3.2 Smart contract

Il contratto ***BakeryShop***, scritto in linguaggio Solidity e definito nel file BakeryShop.sol, è stato scritto e testato su *RemixIDE*, un ambiente di sviluppo online per contratti smart su Ethereum. Lo smart contract implementa la logica on-chain per la gestione di un negozio virtuale di prodotti da forno, in cui ogni prodotto

è rappresentato come un token non fungibile (NFT) sulla blockchain Ethereum. L'obiettivo del contratto è consentire agli utenti di acquistare questi prodotti tramite transazioni Ethereum, con alcune funzionalità aggiuntive come la lotteria per premi gratuiti.

Il contratto definisce vari tipi di prodotti da forno, ovvero *Coffee, Croissant, Muffin, Donut, Cupcake* e *Brownie*, ognuno con un limite massimo di nove unità per tipo. La struttura del contratto è la seguente:

- ***Costruttore***: Quando il contratto viene dispiegato sulla blockchain, il costruttore inizializza il negozio con una quantità iniziale di ciascun tipo di prodotto, impostando il nome, il prezzo e la quantità. Ogni prodotto viene mintato come un NFT e assegnato al proprietario del contratto.

- ***mintProduct****(ProductType _type, string memory _name, uint256 _price, uint256 _quantity)*: Questa funzione è utilizzata per creare nuovi token di un determinato tipo di prodotto. La quantità di nuovi token deve essere valida e non superare il limite massimo per tipo di prodotto. Viene aggiornato il totale mintato e la quantità disponibile di quel prodotto, e nuovi token vengono creati e assegnati al proprietario del contratto.

- ***getTotalProducts()***: Restituisce il numero totale di prodotti disponibili nel negozio, sommando le quantità disponibili di tutti i tipi di prodotto.

- ***getProductInfo****(ProductType _type)*: Restituisce le informazioni dettagliate di un prodotto specifico, inclusi nome, prezzo, quantità disponibile e totale mintato.

- ***purchaseProduct****(ProductType _type, uint256 _quantity)*: Consente agli utenti di acquistare uno o più prodotti della stessa tipologia pagando in ETH. La funzione verifica che il prodotto sia disponibile in quantità sufficiente e calcola il prezzo totale. I token vengono trasferiti dall'owner del contratto all'acquirente, e viene emesso un evento di acquisto. Inoltre, viene effettuata una lotteria per assegnare premi gratuiti agli acquirenti fortunati.

- ***getLotteryResult()***: Chiama la funzione *getRandomNumber(100)* per ottenere un numero casuale tra 0 e 99, e se tale numero è inferiore a 10, la funzione restituisce true (indicando che il risultato della lotteria è positivo), altrimenti, restituisce false (indicando che il risultato della lotteria è negativo). La funzione ***getRandomNumber(uint256 _max)*** ottiene tale numero randomico utilizzando il timestamp del blocco corrente e la difficoltà del blocco per calcolare un valore hash. Il numero che viene restituito è il resto della divisione di questo valore hash per 100.

- ***awardFreeBreakfast****(address _winner)*: Questa funzione assegna un prodotto gratuito a un utente che ha effettuato un acquisto, se ha vinto la lotteria. Utilizza le funzioni *canMintAnyProduct()* per verificare se ci sono prodotti che possono ancora essere mintati e *getLotteryResult* per determinare se l'utente ha vinto la lotteria. Se l'utente ha vinto e ci sono prodotti disponibili, un prodotto viene trasferito all'utente utilizzando la funzione *_transfer()* di ERC721 presente in *anyAvailableProduct()*. Se non ci sono prodotti disponibili o se l'utente non vince la lotteria, viene emesso un messaggio appropriato.

- ***restockProductExternal****(ProductType _type, uint256 _quantity)*: Permette al proprietario del contratto di rifornire un prodotto specifico, aumentando la quantità disponibile e creando nuovi token se necessario.

- ***setTokenURIExternal****(uint256 tokenId, string memory _tokenURI)*: Consente al proprietario di impostare l'URI di un token specifico, aggiornando i metadati del token.

3.3 Integrare lo Smart contract con il Front End

In seguito alla realizzazione del contratto *BakeryShop* è stato poi necessario collegarlo al frontend della nostra applicazione, dal momento che quest'ultimo deve gestire linterfaccia utente e linterazione con la blockchain, utilizzando lestensione browser MetaMask. Lintero frontend è stato sviluppato combinando JavaScript, HTML e CSS. La logica dellapplicazione è contenuta interamente nel file *script.js*.

Per eseguire tale collegamento, sono stati seguiti i seguenti passaggi:

1. **Sviluppo e Compilazione del Contratto in RemixIDE**: Dopo aver completato lo sviluppo, si è utilizzato il compilatore di RemixIDE per generare il bytecode e l'Application Binary Interface (ABI) del contratto. L'ABI è un componente fondamentale poiché definisce l'interfaccia del contratto e consente al frontend di interagire con esso.

2. **Deploy del Contratto su Rete Ethereum Testnet**: Utilizzando MetaMask, un portafoglio digitale che funge da bridge tra RemixIDE e la blockchain di Ethereum, si è dunque distribuito il contratto su Sepolia Testnet, come si vede in 3.2. L'utilizzo di MetaMask ha consentito di firmare e inviare la transazione per il deploy del contratto, ottenendo così l'indirizzo del contratto distribuito che è stato memorizzato nella variabile *const contractAddress* nel codice javascript *script.js*.

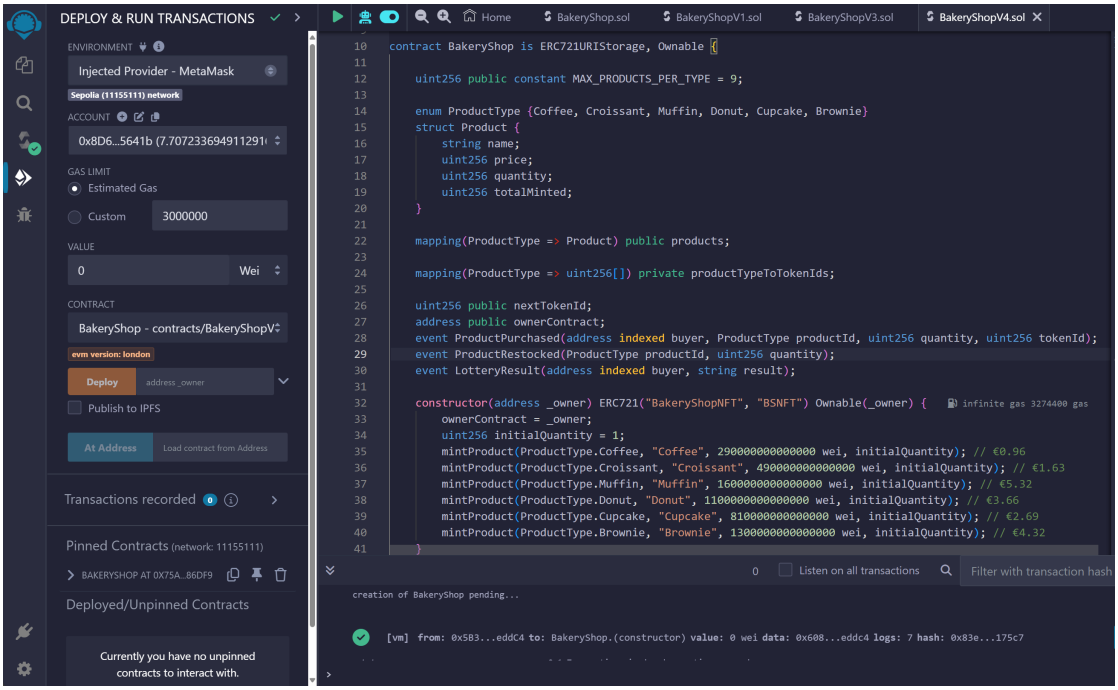


Figura 3.2: La schermata di Deploy in Remix IDE con Metamask come Injected Provider

3. **Integrazione dell'ABI nel Frontend**: Nel progetto frontend, sviluppato in Visual Studio, si è importato l'ABI del contratto. L'ABI, insieme all'indirizzo del contratto distribuito, è stato utilizzato per creare un'istanza del contratto smart (chiamata *myContract*) nel frontend tramite la libreria Web3.js.

4. **Connessione al Contratto con Web3.js**: Si è configurato Web3.js (o Ethers.js) nel progetto frontend per connettersi alla blockchain di Ethereum. Utilizzando l'istanza del contratto creata precedentemente, il frontend è in grado di chiamare le funzioni del contratto, come *purchaseProduct()* o *getProductInfo()*, permettendo agli utenti di interagire direttamente con il contratto tramite l'interfaccia web.

5. **Gestione delle Transazioni con MetaMask**: Quando un utente effettua un'azione che richiede una transazione sulla blockchain, come l'acquisto di un prodotto, viene utilizzata lestensione MetaMask attiva nel browser per chiedere la conferma dell'operazione e firmare la transazione. Questo processo assicura che solo gli utenti autorizzati possano eseguire azioni che modificano lo stato del contratto. Il frontend quindi chiederà allutente il permesso di

collegarsi al proprio account MetaMask, ed ogni transazione (escluse quelle di sola lettura) verso la blockchain dovrà essere validata dallutente nel pop-up apposito dellestensione MetaMask.

```
const buyButtons = document.querySelectorAll('.buy-button');
buyButtons.forEach(button => {
    button.addEventListener('click', async () => {
        const productType = parseInt(button.dataset.productType); // Ottieni il tipo di prodotto dal dataset
        const quantityInput = button.parentElement.querySelector('.buy-quantity');
        const quantity = parseInt(quantityInput.value); // Ottieni la quantità dalla casella di input

        // Richiedi l'account che ha cliccato il bottone
        const accounts = await ethereum.request({ method: 'eth_requestAccounts' });
        const currentPublicAddress = accounts[0];
        console.log("Current public address:", currentPublicAddress);

        //Decommentare la riga successiva per usare la funzione di simulazione
        //simulatePurchase(productType, quantity, currentPublicAddress);

        //Chiamata alla funzione di acquisto con il tipo di prodotto e la quantità desiderata
        purchaseProduct(productType, quantity, currentPublicAddress).then(() => {
            // Aggiorna le info dei prodotti
            updateProductInfo(myContract).then(() => {console.log("Product info updated")} );
        });

    });
});
```

Figura 3.3: Snippet di codice per la connessione con Metamask.

6. **Visualizzazione delle Risposte nel Frontend**: Infine, le risposte delle funzioni del contratto, come l'esito della lotteria o l'aggiornamento delle quantità di prodotto, vengono visualizzate nel frontend, ad esempio tramite opportune finestre di dialogo. Questo feedback in tempo reale offre un'esperienza utente interattiva e coinvolgente.

Questo flusso di lavoro ha permesso di integrare efficacemente il contratto *BakeryShop* con un'interfaccia utente web, rendendo possibile per gli utenti interagire con il contratto in modo intuitivo e sicuro.

4. Esempio d'uso

Nel seguente capitolo viene riportato brevemente quello che potrebbe essere il tipico workflow dellapplicazione web tramite una descrizione testuale e lausilio di immagini.

Come già illustrato precedentemente, la Web DApp *Bakery Shop NFT* permette agli utenti di creare e acquistare prodotti unici di una panetteria digitale, rappresentati come NFT conformi allo standard ERC-721. La piattaforma è stata progettata per offrire un'esperienza semplice e accessibile anche agli utenti non esperti di blockchain.

Una volta che viene avviato il server locale tramite l'estensione *Live Server* di Visual Studio, verrà aperta la schermata di autenticazione di Bakery Shop NFT.

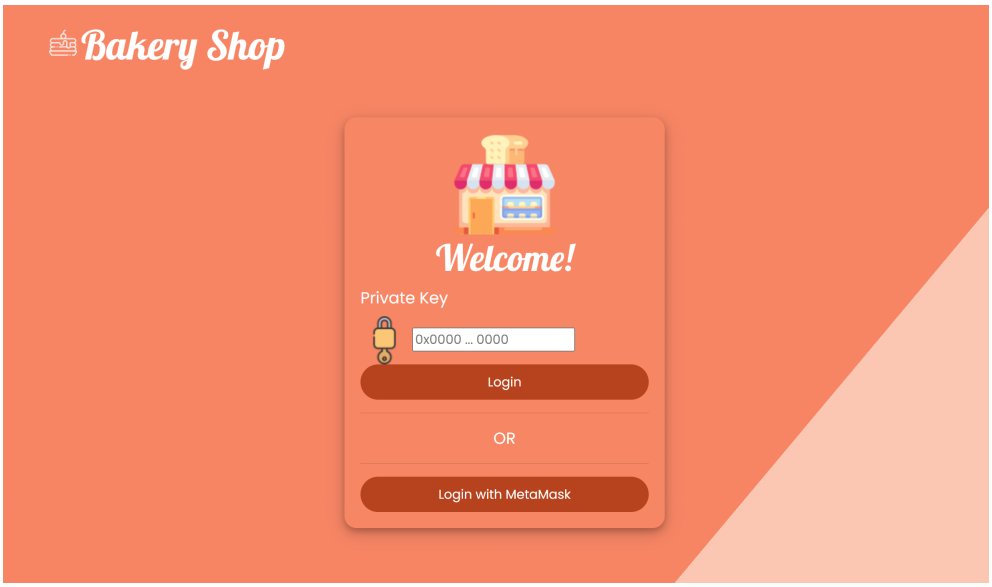


Figura 4.1: Schermata iniziale di autenticazione.

Come si può vedere nella figura 4.1, si ha la possibilità di scegliere fra due alternative per accedere alla homepage:

1. **Accesso tramite Chiave Privata**: In questo caso si può inserire la chiave privata nel campo apposito e fare clic su Login.

2. **Accesso tramite MetaMask**: Se si preferisce una soluzione più user-friendly e rapida, si può scegliere di collegarsi direttamente tramite MetaMask. E' necessario fare clic su Login with MetaMask e seguire le istruzioni per collegare il tuo portafoglio MetaMask allapplicazione. Si aprirà una schermata visibile a quella in figura 4.2.

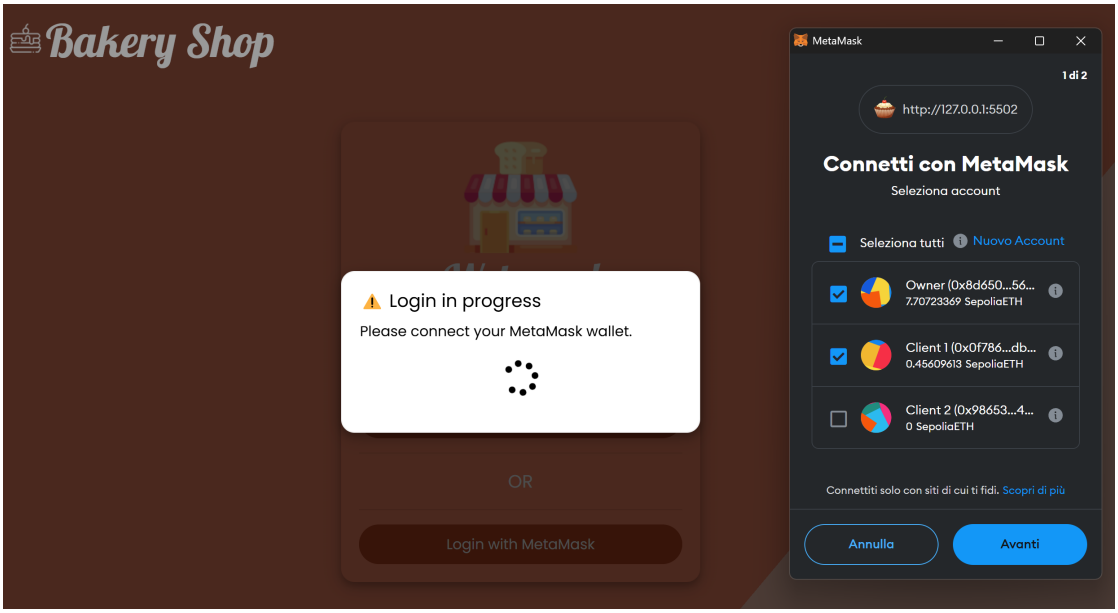


Figura 4.2: Richiesta di autenticazione tramite Metamask.

Una volta completata l'autenticazione, si aprirà la seguente homepage in figura 4.3.

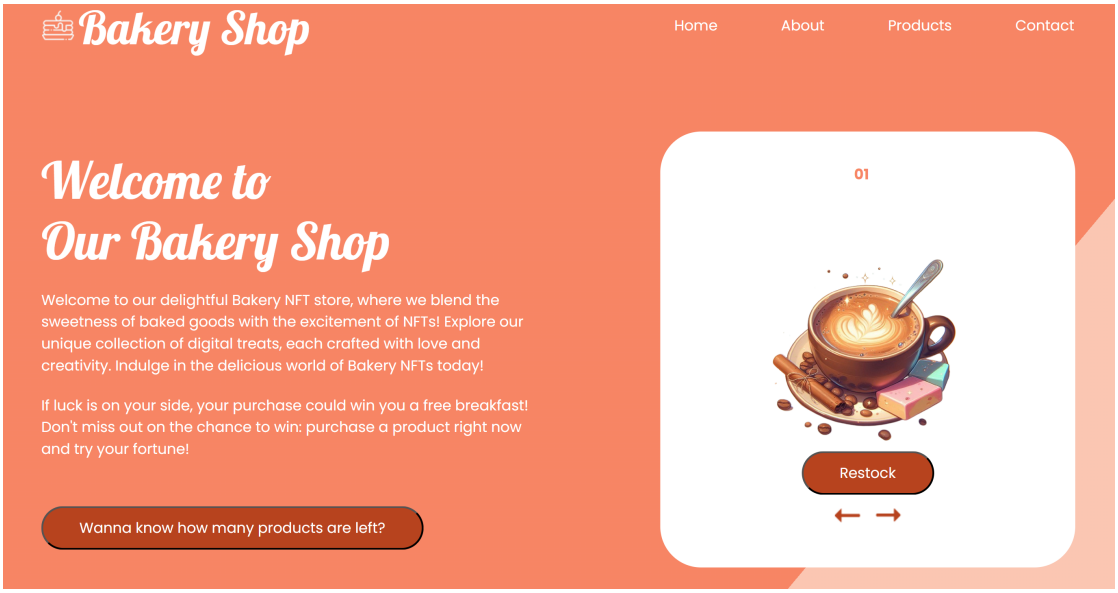


Figura 4.3: La homepage della DApp *BakeryShop*

La homepage dell'applicazione Bakery Shop è strutturata in diverse sezioni, ciascuna con funzioni specifiche per migliorare l'esperienza dell'utente. Nell'angolo in alto a destra della homepage è possibile cliccare negli appositi pulsanti per esplorare più rapidamente le sezioni di proprio interesse. Di seguito tali sezioni e funzionalità verranno analizzate singolarmente.

4.1 Sezione Home: Totale dei prodotti e Restock

La sezione "*Home*" dà il benvenuto agli utenti nel Bakery NFT store. Al centro della sezione, c'è un pulsante con scritto *"Wanna know how many products are left?"* che, se cliccato, apre una finestra di dialogo che mostra la quantità attuale di prodotti disponibili come si vede in figura 4.4. Questa funzione permette agli utenti di monitorare in tempo reale la disponibilità dei prodotti.

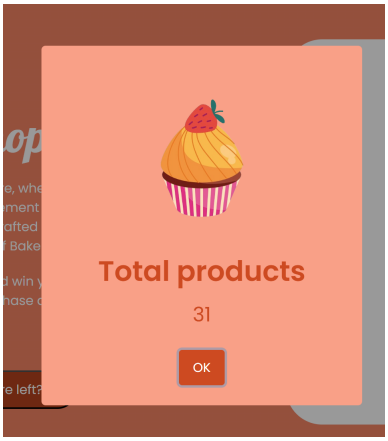
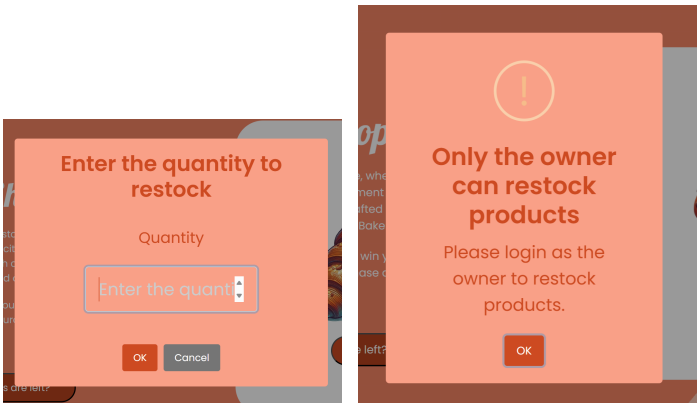


Figura 4.4: Il dialog che mostra il totale dei prodotti presenti in quel momento nella Bakery.

La sezione inoltre include un carosello a destra che mostra le sei tipologie di prodotti disponibili, ognuno accompagnato da un pulsante "*Restock*". Il pulsante "Restock" è abilitato solo per il proprietario del contratto, e se un normale utente tenta di utilizzarlo, appare un dialogo di avviso che informa l'utente di non avere i permessi necessari, come mostrato in figura 4.5b.

Invece quando il pulsante "*Restock*" viene cliccato dall'owner della Bakery, appare un dialog che chiede la quantità di prodotti che si desidera rifornire 4.5a.

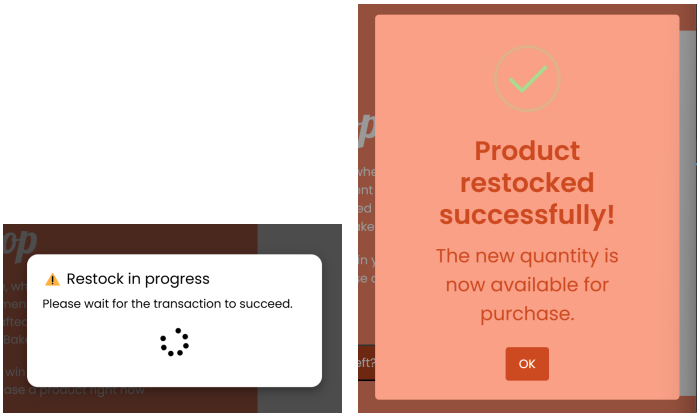


(a) (b)

Figura 4.5: I diversi dialog che compaiono se si clicca il bottone *restock* a seconda che lo faccia l'owner (a) o il client (b).

Successivamente, viene richiesto tramite Metamask il permesso per avviare la transazione riguardante il *restock* sulla blockchain. Durante il processo di transazione, viene visualizzato un dialog di attesa per il completamento della stessa

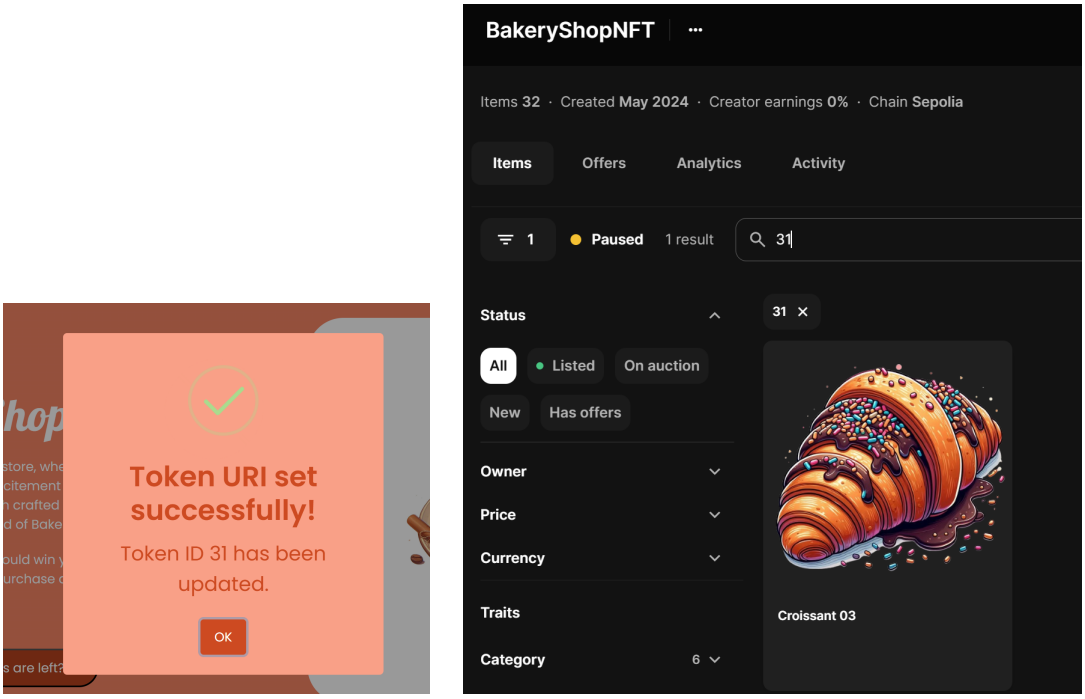
4.6a. Se la transazione va a buon fine, appare un dialog che informa l'owner che il restock è avvenuto con successo 4.6b.



(a) (b)

Figura 4.6: I due dialog che compaiono durante l'attesa della transazione (a) e a seguito di un restock avvenuto con successo (b).

Subito dopo, compare un'altra richiesta di conferma transazione tramite Metamask, questa volta per impostare l'URI corrispondente ai prodotti riforniti, al fine di associare loro i metadati previsti. Una volta confermata e completata con successo questa seconda transazione, appare un dialog che informa che l'URI è stato correttamente impostato per i token riforniti, indicando anche il relativo ID del token 4.7a.



(a) (b)

Figura 4.7: Il dialog che conferma il set dell'URI per il token (a) e i relativi metadati associati a quel token su OpenSea. (b).

Infine, se si cerca il contratto della bakery su OpenSea Testnet e si individuano i token ID riforniti, si potranno visualizzare i metadati associati a quei token il cui URI è stato impostato 4.7b. Questo permette di verificare che i prodotti riforniti siano correttamente registrati e visibili sulla piattaforma OpenSea Testnet.

4.2 Sezione Products

La sezione "*Products*" è dedicata alla presentazione delle sei tipologie di prodotti disponibili, riportata in figura 4.8. Le sei categorie di prodotti sono:

1. Coffee
2. Croissant
3. Muffin
4. Donut
5. Cupcake

6. Brownie

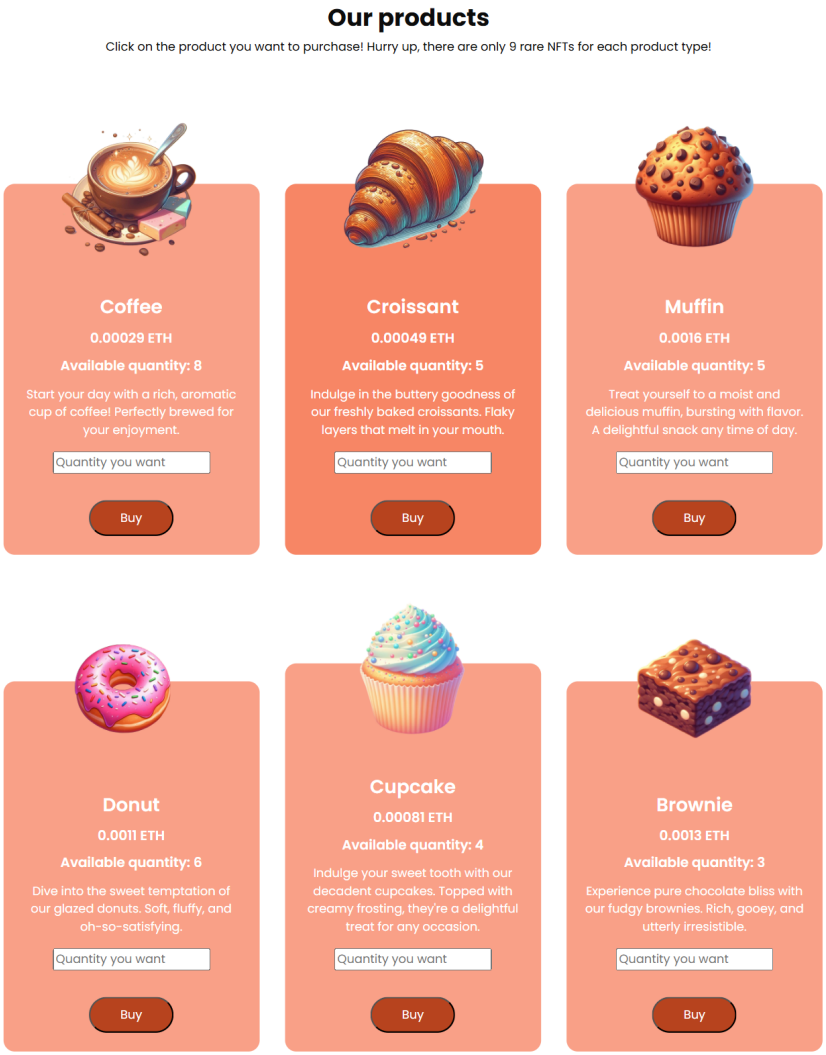


Figura 4.8: I prodotti presenti nella Bakery *BakeryShop*

Ogni prodotto è accompagnato da un pulsante "*Buy*" che permette agli utenti di acquistare il prodotto desiderato, nella quantità specificata nel placeholder *"Quantity you want"*. Una volta scritta la quantità che si desidera acquistare, cliccando su "*Buy*" si apre la finestra Metamask che chiede conferma prima di inviare la transazione apposita alla blockchain e che riporta i costi previsti, comprensivi di Fees oltre che del prezzo del prodotto selezionato, come si vede in figura 4.9. Questo rende l'interfaccia user-friendly e facilita il processo di acquisto.

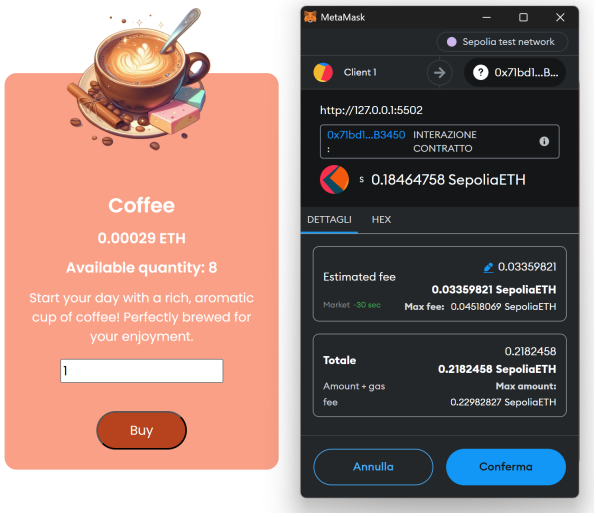


Figura 4.9: Pop-up richiesta Metamask di permesso di acquisto di un Caffè.

Durante l'attesa per il completamento della transazione sulla blockchain, viene mostrato il seguente dialog con uno spinner, in figura 4.10.

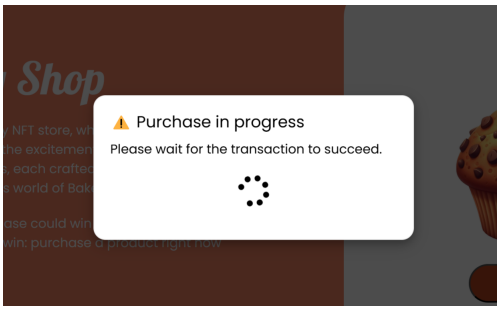
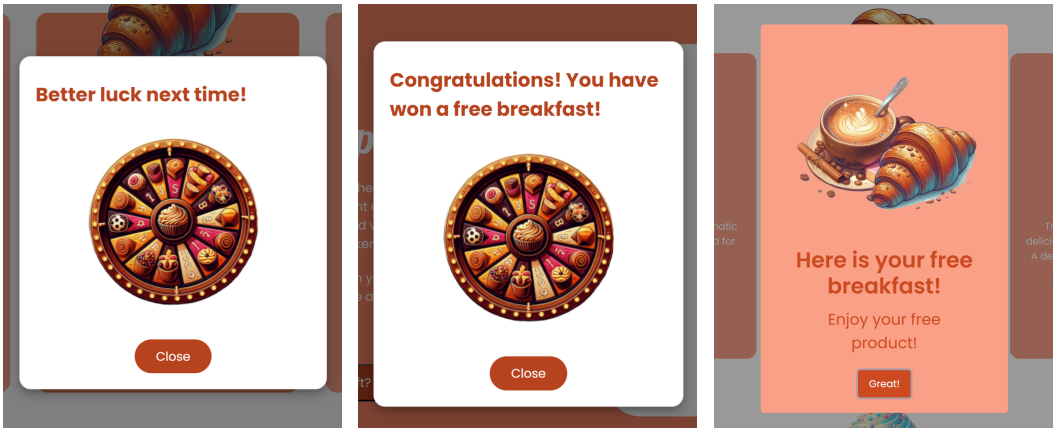


Figura 4.10: Dialog per l'attesa del completamento della transazione di *purchase* del prodotto.

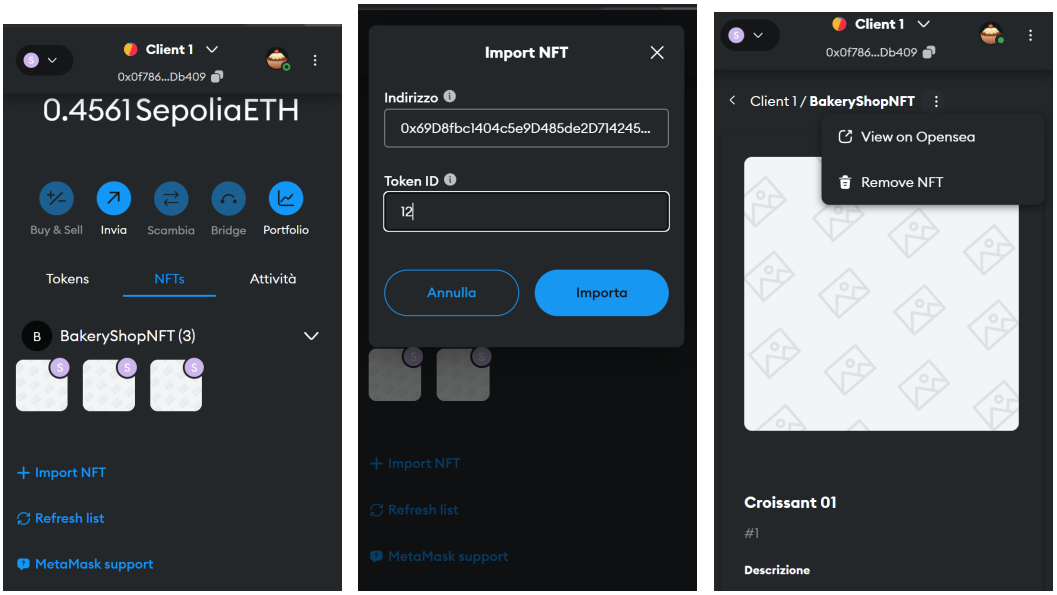
A seguito del completamento della transazione, se essa è andata a buon termine, l'utente partecipa automaticamente a una lotteria randomica. Se l'utente vince la lotteria, riceve gratuitamente un prodotto disponibile come bonus. In caso di vittoria, appare un dialog che notifica l'utente della vincita, come mostrato in figura 4.11b ed in seguito appare un ulteriore dialog che lo informa di aver vinto un prodotto NFT bonus 4.11c. In caso di sconfitta, viene mostrato un dialog che informa l'utente di non aver vinto, come mostrato in figura 4.11a.



(a) (b) (c)

Figura 4.11: Tre schermate dell'applicazione Bakery Shop. Le schermate (a) e (b) mostrano l'esito della lotteria in un dialog che appare a seguito di un acquisto. La schermata (c), nel caso di vittoria, notifica all'utente di aver vinto un prodotto bonus.

Dopo aver acquistato un NFT, è possibile visualizzarlo nel proprio wallet Metamask seguendo questi semplici passaggi. Prima di tutto, occorre cliccare sull'estensione Metamask del proprio browser per aprire il wallet. Andare nella sezione "NFT" e selezionare "*Import NFT*" 4.12a. Si aprirà una finestra in cui sarà necessario inserire l'indirizzo del contratto (Contract Address) e l'ID del token appena acquistato o posseduto 4.12b. Una volta inseriti questi dati, il token verrà aggiunto al wallet e sarà visibile nella sezione NFT.



(a) (b) (c)

Figura 4.12: Tre schermate del Wallet Metamask in cui trovare gli NFT posseduti dall'account.

Per vedere ulteriori dettagli sul token NFT, come i metadati e l'immagine associati, è necessario cliccare sul prodotto NFT all'interno del wallet e selezionare "*View on OpenSea*" dal menu a tendina del token 4.12c. Questo indirizzerà l'utente alla pagina di OpenSea Testnet dove sarà possibile visualizzare il token e i suoi metadati, inclusi attributi come Filling e Topping 4.13.

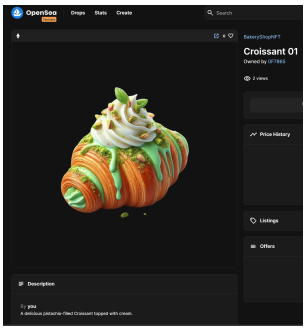


Figura 4.13: Il token e i suoi metadati associati visibili su OpenSea.

Su OpenSea Testnet, sarà inoltre possibile visualizzare tutti gli altri prodotti mintati dalla Bakery fino a quel momento, occorre solo cercare l'indirizzo del contratto 4.14.

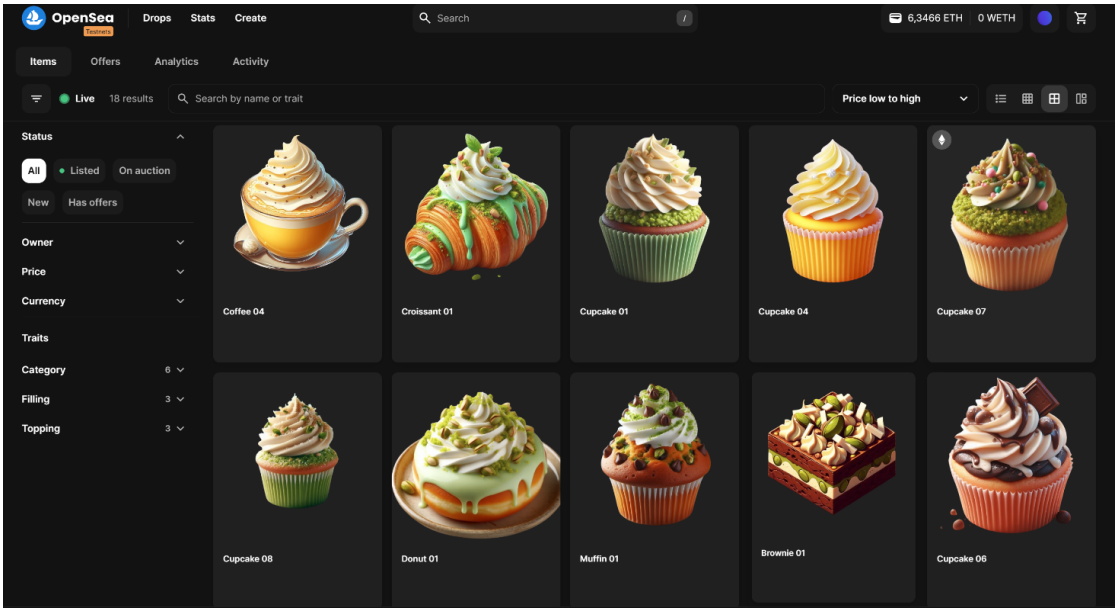


Figura 4.14: Visualizzazione dei prodotti realizzati dalla Bakery su OpenSea Testnet

4.3 Sezione About

La sezione "*About*" offre una breve descrizione del negozio per spiegare che non si tratta di una normale panetteria, ma di un'esperienza digitale sulla blockchain, come si vede in figura 4.15. Qui si sottolinea infatti che i prodotti sono degli NFT unici e certificati sulla blockchain di Sepolia. Si invita inoltre gli utenti a saperne di più sugli NFT cliccando su un pulsante che rimanda alla guida ufficiale degli ERC721 NFT [2].

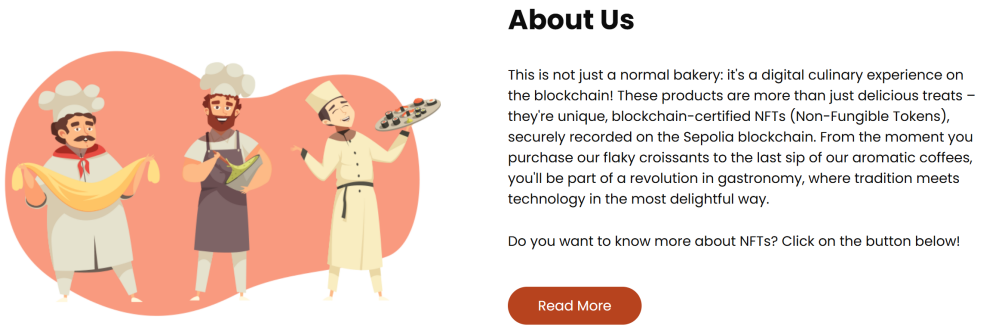


Figura 4.15: Sezione "About us".

4.4 Sezione Latest Clients

La sezione "*Latest Clients*" consiste in un carosello che mostra gli ultimi sei utenti che hanno effettuato un acquisto. Per ogni utente, vengono visualizzati l'indirizzo del portafoglio, il prodotto acquistato e la quantità. Se non ci sono ancora acquisti, viene mostrato un messaggio di default, come si vede nel riquadro a destra nell'immagine 4.16. Questo carosello consente di tenere traccia delle transazioni recenti e offre una panoramica trasparente e aggiornata delle attività dei clienti nel negozio NFT.

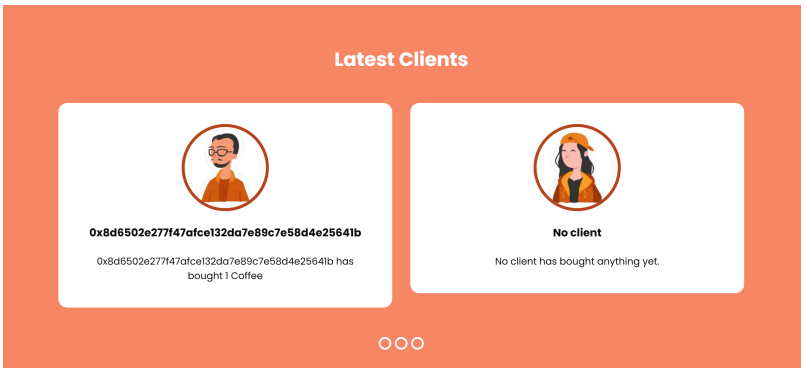


Figura 4.16: Sezione "*Latest Clients*"

4.5 Sezione Contact

La sezione "*Contact*" si trova nella parte finale della pagina web. Essa fornisce le informazioni di contatto essenziali dell'università per la quale è stato realizzato tale progetto, includendo un indirizzo email, un numero di telefono e l'indirizzo 4.17. Inoltre riporta il nome della creatrice del progetto.

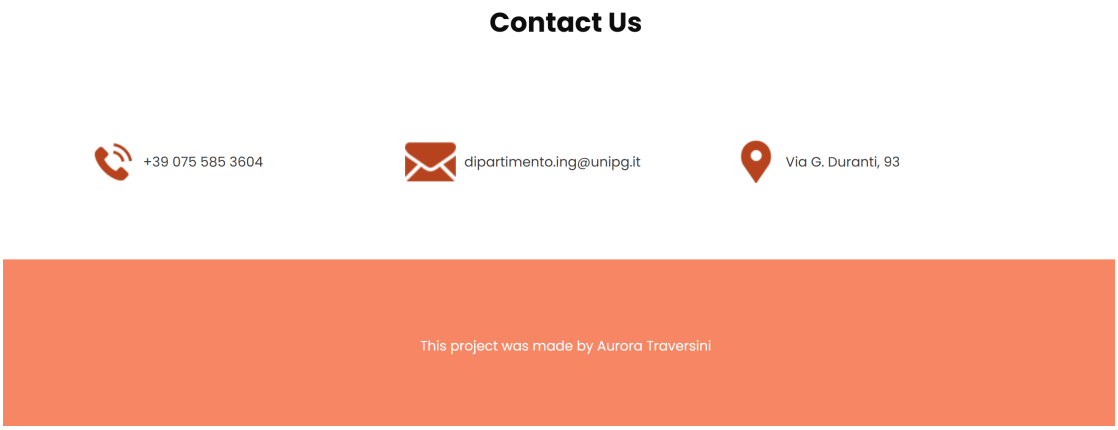


Figura 4.17: Sezione "Contact"

5. Problemi Riscontrati

Si riportano di seguito i principali problemi riscontrati nello svolgimento del progetto *BakeryShop*.

- **Cambiamento del tipo di token utilizzato:** Inizialmente, il progetto prevedeva la creazione di token ERC20, quindi token fungibili. Tuttavia, si è deciso di passare ai token NFT (ERC721) perché risultavano più interessanti e stimolanti, oltre che più coerenti con l'idea di realizzare prodotti unici e collezionabili. Questo cambiamento ha aggiunto ulteriore complessità, poiché gli NFT fanno riferimento ad uno standard diverso rispetto ai comuni token fungibili. Inoltre, essi richiedono la gestione di metadati univoci per ogni singolo token, aumentando la complessità della struttura del contratto e dell'interazione con la blockchain.

- **Unicità degli NFT:** Assicurarsi che ogni NFT fosse univoco ha richiesto la creazione individuale dei metadati per ogni prodotto. Ogni tipo di prodotto ha un proprio JSON e una propria immagine, caricati manualmente sulla piattaforma Pinata. Questo processo è stato laborioso e soggetto a errori, richiedendo un attento controllo di ogni file caricato per garantire la correttezza e la coerenza dei dati. Per approfondire le scelte fatte per i metadati, si rimanda all'Appendice 8.

- **Assegnazione degli URI corretti durante il restock:** Garantire che ogni token ricevesse l'URI corretto al momento del restock è stato complesso. La funzione di restock doveva non solo creare nuovi token ma anche assegnare loro l'URI appropriato, mantenendo la coerenza con i dati già esistenti e assicurando che ogni nuovo token fosse correttamente collegato al metadato e all'immagine corrispondente.

- **Limitazione della quantità massima:** Un'altra sfida è stata introdurre successivamente una limitazione del numero massimo di token per ogni tipologia di prodotto. Questo ha richiesto una gestione accurata del conteggio

dei token emessi per ogni tipo di prodotto, per evitare che si superasse il limite massimo stabilito, aggiungendo complessità alla logica del contratto.

6. Sviluppi futuri e Conclusioni

L'applicazione realizza esaustivamente quanto previsto nella sezione 2, implementando una semplice ma efficace DApp per interagire con la Blockchain e per la creazione di NFT.

Tuttavia, esistono diverse aree di miglioramento e potenziali sviluppi che potrebbero arricchire ulteriormente l'applicazione:

- La possibilità di aggiungere nuovi prodotti e aggiornare le quantità esistenti.
- Superare il limite massimo di 9 unità per ogni tipo di prodotto, stabilito per facilitare la creazione dei corrispondenti json per i metadati.
- Si potrebbe complicare ulteriormente la logica della lotteria. Inoltre si potrebbe permettere ad un utente di essere messo in "lista di attesa" per ricevere una reward futura nel caso in cui si fosse superato il limite massimo di 9 prodotti per tutte le tipologie presenti.
- Aggiunta della possibilità per gli utenti di selezionare e acquistare prodotti di tipi diversi nello stesso acquisto, implementando un carrello virtuale. Questa funzionalità migliorerà l'esperienza di acquisto, rendendola più flessibile e conveniente
- Implementare programmi di fidelizzazione per premiare i clienti abituali con sconti o prodotti gratuiti.
- Implementare oracoli per ottenere dati esterni (relativi all'andamento della cryptovaluta), per aggiornare dinamicamente i prezzi dei prodotti.

In conclusione, il progetto sviluppato *BakeryShop* offre un modello completo per la vendita e la gestione di prodotti da forno su blockchain, utilizzando NFT per rappresentare ogni prodotto. L'integrazione con MetaMask consente agli utenti di effettuare acquisti in modo sicuro e trasparente. La funzionalità di lotteria

aggiunge un elemento di *gamification*, aumentando l'interesse e l'engagement degli utenti. L'uso della tecnologia blockchain garantisce che tutte le transazioni siano tracciabili e immutabili, offrendo maggiore sicurezza sia per i clienti che per i proprietari, dal momento che la gestione degli acquisti attraverso contratti intelligenti riduce il rischio di frodi o discrepanze.

7. Bibliografia

[1] ERC721UriStorage. ERC721URIStorage Non-Fungible Token Standard | docs.openzeppelin.com. [Online https://docs.openzeppelin.com/contracts/4.x/api/token/erc721].

[2] Ethereum. Non-fungible tokens (nft). [Online https://ethereum.org/en/nft/].

[3] Metamask. The ultimate crypto wallet for defi,web3 apps, and nfts. [Online https://metamask.io/].

[4] Pinata. Pinata: Ipfs & farcaster apis. [Online https://www.pinata.cloud/].

[5] Remix-IDE.readthedocs.io. Welcome to remix's documentation! remix - ethereum ide 1 documentation. [Online https://remix-ide.readthedocs.io/en/latest/].

[6] ChainSafe Systems. Web3.js — Javascript Ethereum API, 2024. [Online https://web3js.org/; controllata il 24-gennaio-2024].

[7] Moralis Web3 Technology. Sepolia testnet guide what is the sepolia testnet? [Online https://moralis.io/sepolia-testnet-guide-what-is-the-sepolia-testnet/].

[8] OpenSea Testnet. Opensea, the largest nft marketplace. [Online https://testnets.opensea.io/].

[9] Jacob Evans William Entriken, Dieter Shirley and Nastassia Sachs. ERC-721 Non-Fungible Token Standard | ethereum.org, 2018. [Online; controllata il 29-aprile-2024].

8. Appendice

8.1 Scelta dei metadati per rendere unici i token NFT

Trattandosi di NFT, ciascun prodotto vanta un carattere di unicità e rarità. Infatti sono disponibili non più di nove esemplari per ogni categoria, raggiungendo quindi complessivamente 54 pezzi distinti.

Lesclusività è ulteriormente rafforzata dallimpiego di metadati singolari per ciascun NFT, assicurando che ogni pezzo sia non solo limitato in quantità ma anche unico nel suo genere e soprattutto distinguibile rispetto ai prodotti della medesima categoria.

Tali metadati di ciascun NFT sono dei file JSON che sono stati precedentemente memorizzati su Pinata. Essi includono informazioni specifiche sul prodotto, come ad esempio il nome, la descrizione, un URL che punta all'immagine associata al prodotto (ospitata nuovamente su Pinata). Ogni file JSON ha un hash IPFS univoco, generato da Pinata, che viene utilizzato come URI per il token. Dunque all'interno dell'applicazione si ha che ogni volta che viene creato un nuovo prodotto mediante l'apposita funzione *restockProduct()*, viene caricato su Pinata il corrispondente file JSON contenente i metadati di quel prodotto. Per consentire ciò, l'URL del file JSON viene associato al token ID del prodotto tramite la funzione *setTokenURI* dello smart contract ERC-721.

In particolare gli attributi che una volta combinati permettono di specificare ciascun prodotto sono i seguenti:

- **Category**: La categoria del prodotto, ve ne sono dunque sei (ad esempio, *Brownie, Coffee, Donut*).
- **Filling**: Il tipo di ripieno del prodotto. Può assumere tre valori: *Pistachio, Chocolate, Custard*.

- **Topping**: Il tipo di guarnizione del prodotto Può assumere tre valori: *Sprinkles, Coconut, Cream.*

Per garantire l'unicità dei prodotti all'interno della piattaforma *Bakery Shop NFT*, è stato deciso di limitare il numero di prodotti per ogni categoria a 9. Inoltre, sono stati scelti i suddetti due attributi aggiuntivi, *Filling* e *Topping*, ciascuno con tre valori possibili, per assicurare che anche ogni prodotto all'interno di una categoria sia univoco.

Difatti, combinando le categorie con i due attributi, il numero totale di combinazioni C possibili è $C = 6 * 3 * 3 = 54$. Tale numero di combinazioni è sufficiente per distinguere univocamente ogni prodotto nella piattaforma *Bakery Shop NFT*, garantendo che ogni prodotto possa essere identificato in modo univoco non solo tra le diverse categorie ma anche all'interno di ciascuna categoria.